Mission 005 Product Handoff

What I did

• Defined the UX/UI direction for the read-only dashboard and latest mission view after the architecture decision was fixed.

• Clarified the user-facing story for the DB-backed model: the app is read-only, the database is canonical, and freshness is always explicit.

• Wrote recommended wording for freshness, lag, updated-at, and empty/partial/delayed/stale states.

• Captured frontend UI state rules that must be preserved even as the backing source changes.

UX/UI direction

Dashboard: read-only overview

• Present the dashboard as a read-only status surface, not an editor or control panel.

• Keep the latest mission visually dominant so users immediately see the current story.

• Surface freshness and provenance in the header area, not buried in secondary metadata.

• Keep the list of other missions as supporting context, with clear timestamps and status badges.

• Avoid any UI language that implies the user is editing, publishing, or syncing state.

• Do not expose JSON as a primary concept in the main UI; keep it out of the user story except in export/debug contexts.

Latest mission view: narrative replay

• Frame the latest mission as a single chronological story: who acted, what changed, and why it matters.

• Preserve the current one-timeline mental model; the user should not have to switch between views to understand the latest state.

• Keep chronology, parallel activity, and step ordering visible.

• Keep freshness visible at the top of the view and repeat updated-at context near the mission metadata.

• If the mission is partial, delayed, or stale, communicate that in-place with explanatory copy rather than hiding the timeline.

Recommended label copy

Freshness badges

• `Live` for fresh data.

• `Delayed` for data that is still usable but lagging.

• `Stale` for data that is too old to trust as fully current.

• `Partial feed` for incomplete story data.

• `No mission data yet` for empty state.

Updated-at / lag copy

Use the absolute timestamp plus a short lag phrase. Keep both visible when possible.

Recommended patterns:

• `Updated May 27, 5:24 PM`

• `Updated May 27, 5:24 PM · 2 min behind live clock`

• `Updated May 27, 5:24 PM · stale by about 18 min`

• `Updated May 27, 5:24 PM · some fields are still missing`

Preferred terminology:

• Use `Updated at` or `Updated` for the timestamp label.

• Use `Lag` only in technical/tooling contexts; in the UI, prefer human phrasing like `behind live clock`.

• Use `live clock` rather than `JSON`, `feed`, or `source file` in user-facing copy.

• If the app cannot calculate lag, say `Lag unavailable` or `Updated at unavailable` explicitly.

Empty / partial / delayed / stale states

• **Empty:** `No mission data yet. Waiting for the first canonical mission to appear.`

• **Partial:** `Partial feed. Some fields or steps are still missing.`

• **Delayed:** `Delayed. This view is behind the live clock but still usable.`

• **Stale:** `Stale. This view is older than the freshness threshold.`

• **Unavailable metadata:** `Updated at unavailable` or `Lag unavailable`, never blank.

UI state rules the frontend must preserve

• The UI must remain read-only for end users; no mutation actions, write affordances, or edit controls.

• The frontend must continue consuming the existing dashboard and latest-mission read contracts without changing the response shape.

• Freshness state must remain explicit and visible in both the dashboard and latest mission view.

• `updatedAt` and lag indicators must not be hidden when data is available.

• Empty, partial, delayed, and stale states must render as distinct, legible states rather than collapsing into one generic error.

• The UI must preserve chronology and event ordering, including visible parallel work groups.

• The latest mission view should always identify the currently selected/latest mission clearly.

• If data is incomplete, render the available story and label the gaps; do not invent missing content.

• Do not infer truth from JSON filenames, source URLs, or local fixtures.

• Keep the dashboard shape stable even when the source changes from JSON-backed data to DB-backed data.

Presentation-level open questions

These are styling and copy choices only; they do not affect architecture.

- Should the freshness badge use `Live` or `Fresh` as the primary label in production UI?
- Should the dashboard emphasize absolute time first or lag first in the metadata line?
- Should stale state use an amber warning tone or a stronger red tone?
- Should the latest mission header say `Updated at` or `Last updated`?
- Should the empty state mention the canonical database explicitly, or keep the copy user-neutral?

Product handoff summary

Mission 005 should read as a stable, read-only, DB-backed dashboard with explicit freshness cues. The user-facing story must stay simple: the latest mission is the main narrative, the rest of the dashboard is supporting context, and every state must make freshness and completeness obvious.